

HTML od podstaw: podręcznik uczestnika

Accessibility First · materiał do zajęć „Nauka HTMLa od podstaw” · wersja z 3.10.2026

Ten podręcznik zbiera to, co omawiamy na zajęciach. Nie zastępuje dokumentacji. Ma pomóc wrócić do tematu po zajęciach i naprawić własną stronę.

Każdy rozdział ma ten sam układ: **co się psuje, reguła, kod źle i dobrze, jak sprawdzić.**

Przykłady działają po wklejeniu do pustego pliku albo do placu zabaw na a11yfirst.pl/plac-zabaw-html.

Uwaga o czytnikach ekranu. Tam, gdzie piszemy, co „ogłosi czytnik”, to przybliżenie. NVDA, JAWS i VoiceOver mówią to inaczej i zależnie od ustawień. Pewne jest to, jaką rolę, nazwę i stan dostaje element. To widać w drzewie dostępności w narzędziach deweloperskich przeglądarki.

Spis treści

[1. Jak zbudowana jest strona](#)

[2. Skąd się wziął HTML i po co semantyka](#)

[3. Nagłówki i landmarki](#)

[4. Obrazki](#)

[5. Link czy przycisk?](#)

[6. Div i span](#)

[7. Listy](#)

[8. Tabele](#)

[9. Formularze](#)

[10. Fokus, kolor, rozmiar](#)

[11. Przykład z zajęć: recenzja strony](#)

[12. Jak sprawdzić własną stronę w 10 minut](#)

[Ściąga](#)

[Źródła](#)

1. Jak zbudowana jest strona

HTML jak człowiek

- **Głowa** = `<head>`. Myśli, których nie widać: tytuł karty, język, kodowanie, opis strony.
- **Ciało** = `<body>`. Wszystko, co widać i z czym wchodzimy w interakcję.
- **CSS** = **makijaż**. Wygląd: kolory, czcionki, układ. Nie zmienia tego, czym element jest.
- **JavaScript** = **humory**. Reakcje: kliknięcia, rozwijanie, podmiana treści.

Minimalny poprawny dokument

HTML

```
<!DOCTYPE html>
<html lang="pl">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Kontakt | Biblioteka Miejska w Opolu</title>
</head>
<body>
  <h1>Kontakt</h1>
  <p>Zapraszamy od poniedziałku do piątku.</p>
</body>
</html>
```

- `<!DOCTYPE html>`: bez niego przeglądarka przechodzi w tryb zgodności wstecznej i inaczej liczy wymiary.
- `lang="pl"`: czytnik ekranu wybiera polski głos i wymowę. Bez tego może czytać polski tekst z angielskim akcentem (WCAG 3.1.1 Język strony).
- `<title>`: pierwsza rzecz, którą słyszy użytkownik czytnika po wejściu na stronę. Najpierw to, co konkretne, potem nazwa serwisu (WCAG 2.4.2 Tytuł strony).

Gdzie wstawić CSS i JavaScript

- CSS w `<style>` w `<head>` albo w osobnym pliku przez `<link rel="stylesheet">`.

- Skrypt najlepiej na końcu `<body>`. Skrypt widzi tylko to, co stoi nad nim, więc musi być **pod** elementami, których szuka.
- `<script>` jest niewidoczny dla czytnika ekranu. Nie zmienia struktury strony.

Dwa błędy, które „zjadają” pół strony

1. **Niedomknięty `<style>`**. Wszystko po nim przeglądarka traktuje jako CSS. Znika treść, choć kod wygląda na kompletny.
2. **Brakująca klamra `}` w CSS**. Przeglądarki obsługują dziś zagnieżdżanie reguł (CSS nesting). Bez klamry kolejne reguły trafiają do środka poprzedniej i przestają działać. Żaden błąd się nie pojawia, styl po prostu znika.

Podobnie działa **nieistniejąca zmienna**: `color: var(--bład)` bez zdefiniowanego `--bład` nie zgłasza błędu, tylko po cichu nie działa.

2. Skąd się wziął HTML i po co semantyka

- 1989: Tim Berners-Lee w CERN proponuje sieć dokumentów połączonych linkami.
- 1991: pierwszy opis znaczników HTML. Były tytuły, nagłówki, akapity, listy i linki. **Nie było żadnego znacznika wyglądu**: pogrubienia, koloru ani czcionki. Obrazy doszły w 1993 roku.
- 1994: powstaje W3C, organizacja, która rozwija standardy sieci, w tym WCAG.

HTML od początku opisuje, **czym jest** treść, a nie jak wygląda. To jest semantyka.

Co z tego ma użytkownik

Przeglądarka zamienia HTML w **drzewo dostępności** (accessibility tree). Każdy element dostaje w nim:

- **rolę**: czym jest (przycisk, link, nagłówek, lista),
- **nazwę**: jak się nazywa (tekst przycisku, etykieta pola, alt obrazka),
- **stan**: w jakim jest stanie (zaznaczony, rozwinięty, nieprawidłowy).

Czytnik ekranu, sterowanie głosem i nawigacja klawiaturą korzystają z tego drzewa, a nie z wyglądu strony. `<div>` wystylowany na przycisk wygląda jak przycisk tylko dla osoby, która go widzi.

ARIA w jednym akapicie

ARIA (Accessible Rich Internet Applications) to atrybuty, które zmieniają to, co trafia do drzewa dostępności: `role`, `aria-label`, `aria-expanded` i inne. **ARIA zmienia tylko to, co słyszy czytnik. Nie dodaje fokusu, obsługi klawiatury ani zachowania.**

Pierwsza zasada ARIA według W3C: jeśli istnieje natywny element HTML, który robi to, czego potrzebujesz, użyj go.

3. Nagłówki i landmarki

Co się psuje. Użytkownik czytnika ekranu nie przegląda strony wzrokiem. Skacze po nagłówkach i po obszarach strony. Jeśli nagłówki są tylko „dużą czcionką”, strona dla niego nie ma spisu treści.

Reguła. Nagłówek to tytuł sekcji, nie rozmiar tekstu. Obszary strony oznaczamy elementami `header`, `nav`, `main` i `footer`.

× Źle: wygląda jak nagłówek, nie jest nagłówkiem

HTML

```
<p class="duzy-tytul">Godziny otwarcia</p>
<p>Poniedziałek–piątek, 8:00–18:00</p>
```

✓ Dobrze

HTML

```
<h2>Godziny otwarcia</h2>
<p>Poniedziałek–piątek, 8:00–18:00</p>
```

Dobre praktyki:

- jeden `h1` z tematem strony,
- poziomy bez przeskoków: po `h2` przychodzi `h3`, nie `h5`,
- bez pustych nagłówków. Nagłówek, pod którym nic nie ma, w liście nagłówków wygląda jak sekcja-widmo,

- jeden `main` na stronę,
- dwa elementy `nav` dostają różne nazwy, np. `aria-label="Główna"` i `aria-label="Stopka"`.

Skiplink: link „Przejdź do treści”

Osoba korzystająca z klawiatury nie musi przechodzić Tabem przez całe menu na każdej podstronie.

HTML

```
<body>
  <a class="skiplink" href="#tresc">Przejdź do treści</a>
  <header>...</header>
  <main id="tresc">
    <h1>Kontakt</h1>
  </main>
</body>
```

Link może być schowany wizualnie, ale **musi się pokazać po fokusie**. Nie chowaj go przez `display: none`, bo wtedy Tab do niego nie dojdzie.

Jak sprawdzić: NVDA: `H` następny nagłówek, `1 – 6` nagłówek danego poziomu, `D` następny landmark, `Insert+F7` lista elementów. VoiceOver na Macu: `V0+U` otwiera pokrętko (rotor).

WCAG: 1.3.1 Informacje i relacje · 2.4.1 Możliwość pominięcia bloków · 2.4.6 Nagłówki i etykiety.

4. Obrazki

Reguła. Każdy `<img>` ma atrybut `alt`. Jego treść zależy od tego, **po co** jest obrazek.

RODZAJ OBRAZKA	CO WPISAĆ W <code>ALT</code>	PRZYKŁAD
Informacyjny	to, co przekazuje	<code>alt="Mapa dojazdu: wejście od ulicy Kościuszki"</code>
Dekoracyjny	pusty: <code>alt=""</code>	ozdobny ornament, tło
W linku albo przycisku	dokąd prowadzi albo co robi	<code>alt="Pobierz z App Store: aplikacja Moje ING"</code>
Złożony (wykres, schemat)	krótki opis, pełny opis obok w tekście	<code>alt="Wykres sprzedaży 2026, szczegóły w tabeli poniżej"</code>

- Nie pisz „obrazek przedstawiający...”. Czytnik sam mówi, że to grafika.
- Brak `alt` to co innego niż `alt=""`. Bez atrybutu czytnik może przeczytać nazwę pliku.
- Link z tekstem i ikoną: tekst już jest nazwą linku, więc ikona dostaje `alt=""`.

HTML

```
<a href="https://ing.pl">
  Dowiedz się, co robimy w banku
  
</a>
```

WCAG: 1.1.1 Treść nietekstowa.

5. Link czy przycisk?

Reguła. Zmienia się adres strony: **link** (`<a href>`). Coś dzieje się na stronie: **przycisk** (`<button>`).

	LINK <code><A HREF></code>	PRZYCISK <code><BUTTON></code>
Do czego	przenosi: inna strona, kotwica, plik	działa: otwiera, wysyła, przełącza
Enter	tak	tak
Spacja	nie (przewija stronę)	tak
Czytnik ogłasza	„link”	„przycisk”

- Tekst linku ma mieć sens w kontekście. Osiem razy „czytaj więcej” na liście linków niczego nie mówi (WCAG 2.4.4 Cel łączy w kontekście).
- `<button>` w formularzu domyślnie **wysyła formularz**. Przycisk, który ma tylko coś zrobić na stronie, dostaje `type="button"`.
- Link do pliku: podaj format i rozmiar w treści linku, np. „Pobierz informator (PDF, 2 MB)”.

WCAG: 2.1.1 Klawiatura · 2.4.4 Cel łączy (w kontekście) · 4.1.2 Nazwa, rola, wartość.

6. Div i span

`<div>` i `<span>` nic nie znaczą. To jest w porządku: służą do układu i stylowania. Problem zaczyna się wtedy, gdy udają przycisk, link, nagłówek albo listę.

Div jak button: test 1:1

To przykład z naszej strony ćwiczeniowej:

× Źle: wygląda i brzmi jak przycisk, ale nie działa z klawiatury

HTML

```
<div role="button" tabindex="0" onclick="zapisz()">Zapisz</div>
```

- `role="button"`: czytnik powie „przycisk”,
- `tabindex="0"`: Tab do niego dojdzie,

- ale **Enter i Spacja nic nie robią**, bo `onclick` reaguje tylko na kliknięcie. Osoba korzystająca z klawiatury słyszy przycisk, którego nie da się nacisnąć.

✓ Dobrze

HTML

```
<button type="button" onclick="zapisz()">Zapisz</button>
```

Jedna linia. Rola, fokus, Enter, Spacja, obsługa w formularzu: wszystko jest wbudowane.

A jeśli naprawdę musisz użyć diva

Czasem div narzuca gotowy komponent albo system, którego nie zmienisz. Żeby działał tak samo jak przycisk, potrzebujesz:

HTML

```
<div role="button" tabindex="0" id="zapisz">Zapisz</div>
<script>
  const el = document.getElementById("zapisz");
  el.addEventListener("click", zapisz);
  el.addEventListener("keydown", (e) => {
    if (e.key === "Enter") { e.preventDefault(); el.click(); }
    if (e.key === " ") e.preventDefault(); // Spacja nie przewija strony
  });
  el.addEventListener("keyup", (e) => {
    if (e.key === " ") el.click(); // Spacja działa po puszczeniu
  });
</script>
```

- **Enter** uruchamia przycisk od razu, przy wciśnięciu klawisza.
- **Spacja** uruchamia go po puszczeniu klawisza. Przy wciśnięciu trzeba zablokować przewijanie strony.
- `el.click()` wywołuje ten sam kod co kliknięcie myszą, więc akcja jest w jednym miejscu.

Nawet wtedy div **nie dostanie** kilku rzeczy, które `<button>` ma wbudowane:

- `disabled` : nieaktywny przycisk znika z kolejności Tabu i nie reaguje na kliknięcie. Div trzeba wyłączać ręcznie: `aria-disabled="true"` i sprawdzenie w skrypcie,
- `type="submit"` : wysyłanie formularza, także Enterem w polu tekstowym,
- `name` i `value` wysyłane razem z formularzem.

Wniosek: **jedna linia HTML kontra kilkanaście linii JS do napisania i przetestowania.**

Jak sprawdzić: odłóż mysz. Przejdź Tabem po stronie. Czy dochodzisz do wszystkiego, co klikalne? Czy Enter i Spacja działają na każdym przycisku?

WCAG: 2.1.1 Klawiatura · 4.1.2 Nazwa, rola, wartość.

7. Listy

- `<ul>` : kolejność nie ma znaczenia (składniki, cechy).
- `<ol>` : kolejność ma znaczenie (kroki, ranking).
- Menu nawigacji to też lista linków.

Czytnik ogłasza listę i liczbę pozycji, np. „lista, 3 pozycje”. Dokładne brzmienie zależy od czytnika. Użytkownik od razu wie, ile tego jest, i może listę pominąć.

× Źle: lista udawana myślnikami i `<br>`

HTML

```
<p>- Prostym<br>- Klarownym<br>- Łatwym</p>
```

✓ Dobrze

HTML

```
<ul>
  <li>Prostym</li>
  <li>Klarownym</li>
  <li>Łatwym</li>
</ul>
```

Safari z VoiceOverem może przestać ogłaszać listę, gdy w CSS ma `list-style: none`.
Obejście: `<ul role="list">`.

WCAG: 1.3.1 Informacje i relacje.

8. Tabele

Reguła. Tabela służy do danych. Nigdy do układu strony.

Najważniejsze elementy:

- `<caption>`: tytuł tabeli, czytnik ogłasza go przy wejściu do tabeli,
- `<th scope="col">`: nagłówek kolumny, `<th scope="row">`: nagłówek wiersza,
- `<thead>`, `<tbody>`, `<tfoot>`: nagłówek, dane, podsumowanie.

Dzięki `th` czytnik przy każdej komórce podaje, do której kolumny i wiersza należy.
Zamiast samego „18” użytkownik usłyszy „Audyty w 2026, Ania, 18”.

HTML

```
<div class="tabela-przewijana" role="region" aria-labelledby="t-zespol" t
  <table>
    <caption id="t-zespol">Zespół i audyty w 2026 roku</caption>
    <thead>
      <tr><th scope="col">Imię</th><th scope="col">Rola</th><th scope="co
    </thead>
    <tbody>
      <tr><th scope="row">Ania</th><td>Audytorka WCAG</td><td>18</td></tr>
      <tr><th scope="row">Bartek</th><td>Frontend developer</td><td>7</td>
    </tbody>
    <tfoot>
      <tr><th scope="row">Razem</th><td></td><td>25</td></tr>
    </tfoot>
  </table>
</div>
```

Kontener z `tabindex="0"` pozwala przewijać szeroką tabelę klawiaturą na telefonie, a nazwa pozwala go rozpoznać (WCAG 1.4.10 Dopasowanie do ekranu).

Jak sprawdzić: NVDA: **T** następna tabela, **Ctrl+Alt+strzałki** ruch po komórkach.

WCAG: 1.3.1 Informacje i relacje.

9. Formularze

Formularz to miejsce, gdzie brak semantyki boli najbardziej. Osoba, która nie wie, co wpisać w pole, nie wyśle zgłoszenia.

Etykiety, grupy, podpowiedzi

HTML

```
<label for="email">Adres e-mail</label>
<input id="email" name="email" type="email" autocomplete="email" required
      aria-describedby="email-podp">
<p id="email-podp">Na przykład: jan.kowalski@poczta.pl</p>
```

- **Każde pole ma** `<label>` połączony przez `for` i `id`. Kliknięcie etykiety przenosi do pola, a czytnik czyta etykietę razem z polem.
- **Placeholder to nie etykieta.** Znika przy pisaniu i zwykle ma za słaby kontrast.
- `autocomplete` przy danych osobowych (`name`, `given-name`, `email`, `tel`). Przeglądarka podpowie dane, a to pomaga osobom z trudnościami motorycznymi i poznawczymi (WCAG 1.3.5).
- **Grupy pól** (dane osobowe, wybór jednej z opcji) w `<fieldset>` z `<legend>`. Czytnik poda nazwę grupy przy wejściu do niej.
- **Instrukcja przed polami**, nie pod przyciskiem: „Wszystkie pola są wymagane”.
- **Pole wymagane:** atrybut `required` i słowo „wymagane” albo zdanie na początku formularza. Sama gwiazdka albo sam kolor to za mało.
- **Podpowieź** podpięta przez `aria-describedby`. Czytnik przeczyta ją po etykiecie.

Komunikat błędu, który wszyscy zauważą

Błąd musi być widoczny i słyszalny. Nie może opierać się tylko na kolorze (WCAG 1.4.1 Użycie koloru).

HTML

```
<label for="email">Adres e-mail</label>
<input id="email" type="email" aria-invalid="true"
      aria-describedby="email-bład">
<p id="email-bład" class="komunikat-błedu">
  Błąd: adres musi zawierać znak @, na przykład jan.kowalski@firma.pl.
</p>
```

- `aria-invalid="true"` : czytnik powie, że pole jest nieprawidłowe,
- komunikat podpięty przez `aria-describedby` : czytnik przeczyta go przy polu,
- słowo „Błąd:” w treści: informacja nie zależy od koloru,
- komunikat mówi, **jak poprawić**, a nie tylko, że jest źle (WCAG 3.3.3).

Podświetlenie błędnego pola i jego etykiety w CSS:

CSS

```
:root { --bład: #b00020; --bład-tło: #fdecee; }

form [aria-invalid="true"] {
  border: 3px solid var(--bład);
  background: var(--bład-tło);
}

.pole:has([aria-invalid="true"]) label {
  color: var(--bład);
  font-weight: 800;
}
```

Przy dłuższym formularzu dochodzi **podsumowanie błędów** nad formularzem: lista z linkami do błędnych pól. Po próbie wysłania przenosimy na nie fokus, więc czytnik od razu je odczyta.

Komunikat „wysłano”

Komunikat, który pojawia się bez przeładowania strony, wstawiamy do elementu z `role="status"` . Ten element musi być na stronie od początku, pusty. Czytnik ogłosi tekst, który się w nim pojawi, bez przenoszenia fokusu (WCAG 4.1.3 Komunikaty o stanie).

HTML

```
<div id="status" role="status"></div>
```

aria-hidden: ostrożnie

`aria-hidden="true"` ukrywa element przed czytnikiem. Używaj go tylko do rzeczy, które dla czytnika są zbędne: ozdobnej ikony albo licznika, który zmienia się przy każdym znaku.

Nie ukrywaj widocznego tekstu, takiego jak odpowiedź czy instrukcja. Osoba słabowidząca, która patrzy i słucha jednocześnie, widzi tekst, którego czytnik „nie zna”.

WCAG: 1.3.1 · 1.3.5 Określenie pożądanej wartości · 3.3.1 Identyfikacja błędu · 3.3.2 Etykiety lub instrukcje · 3.3.3 Sugestie korekty błędów · 4.1.2 Nazwa, rola, wartość.

10. Fokus, kolor, rozmiar

- **Widoczny fokus.** Nie usuwaj obrysu (`outline: none`) bez zamiennika. Dobry fokus ma dwa kolory, np. żółty i czarny, żeby był widoczny na każdym tle (WCAG 2.4.7 Widoczny fokus).
- **Kontrast.** Tekst: co najmniej 4,5:1 (duży tekst 3:1). Ramki pól i ikony: co najmniej 3:1 (WCAG 1.4.3, 1.4.11).
- **Nie tylko kolor.** Link w tekście ma podkreślenie, błąd ma słowo „Błąd”, pole wymagane ma słowo „wymagane”.
- **Cel dotyku.** Co najmniej 24 na 24 piksele CSS (WCAG 2.5.8, AA). 44 na 44 to poziom AAA (2.5.5) i dobry nawyk.
- `tabindex` . `0` dodaje element do kolejności Taba. `-1` pozwala ustawić na nim fokus skryptem. **Nigdy wartość dodatnia:** przedstawia kolejność Taba w całej stronie. Nagłówki i akapity nie potrzebują `tabindex` .

11. Przykład z zajęć: recenzja strony

Na zajęciach zbudowaliśmy stronę banku z listą, linkami, tabelą zespołu i formularzem kontaktowym. Tabela i formularz były zrobione dobrze. Recenzja znalazła sześć rzeczy do poprawy, od najważniejszej:

1. **Błędne pole nie robiło się czerwone.** CSS korzystał ze zmiennych `--blad` i `--ok`, których nikt nie zdefiniował. Przeglądarka nie zgłasza takiego błędu. Poprawka: zdefiniować je w `:root`.
2. **Brak `<main>` i skiplinka.** Styl skiplinka był w CSS, ale samego linku na stronie nie było. Poprawka: dopisać link i `<main id="tresc">`.
3. **Pusty nagłówek.** `h3` „Zarejestruj się”, pod którym nic nie było. Poprawka: usunąć.
4. **Div udający przycisk.** Opisany w rozdziale 6.
5. `aria-labelledby` na zwykłym `<div>`. Specyfikacja ARIA 1.2 nie pozwala nadawać nazwy elementowi bez roli, więc część czytników ją pomija. Poprawka: usunąć atrybut albo nadać elementowi rolę.
6. `aria-hidden` na widocznych podpowiedziach. Opisane w rozdziale 9.

Wniosek: większość błędów to nie brak wiedzy, tylko brak sprawdzenia. Dlatego ostatni rozdział jest o testowaniu.

12. Jak sprawdzić własną stronę w 10 minut

1. **Walidator W3C** (validator.w3.org): niedomknięte znaczniki, zduplikowane `id`, literówki w nazwach elementów.
2. **Sama klawiatura.** Tab, Shift+Tab, Enter, Spacja. Czy widać fokus? Czy dochodzisz do wszystkiego? Czy kolejność ma sens?
3. **Lista nagłówków i landmarków.** W NVDA `Insert+F7`, w VoiceOverze `V0+U`. Albo rozszerzenie HeadingsMap.
4. **Drzewo dostępności** w narzędziach deweloperskich przeglądarki (zakładka Accessibility). Czy każdy przycisk i pole ma rolę i nazwę?
5. **Automat** (axe DevTools, WAVE). Znajdzie część problemów, nie wszystkie. Nie oceni, czy alt ma sens ani czy kolejność jest logiczna.

Ściągą

- `lang`, `title`, jeden `main`, skiplink
- nagłówki jako mapa strony, bez pustych i bez przeskoków
- każdy `img` ma `alt`, dekoracyjny ma `alt=""`

- link przenosi, przycisk działa, oba działają z klawiatury
- każde pole ma `label`, błąd jest powiązany z polem i nie opiera się na samym kolorze
- tabela ma `caption` i `th` ze `scope`
- widać fokus, `tabindex` tylko 0 albo -1

Źródła

- W3C WAI, WCAG 2.2 po polsku i angielsku: [w3.org/WAI/standards-guidelines/wcag/](https://www.w3.org/WAI/standards-guidelines/wcag/)
- W3C, „Using ARIA” i „ARIA in HTML”: [w3.org/TR/using-aria/](https://www.w3.org/TR/using-aria/), [w3.org/TR/html-aria/](https://www.w3.org/TR/html-aria/)
- W3C WAI, ARIA Authoring Practices Guide: [w3.org/WAI/ARIA/apg/](https://www.w3.org/WAI/ARIA/apg/)
- W3C WAI Tutorials (obrazy, tabele, formularze): [w3.org/WAI/tutorials/](https://www.w3.org/WAI/tutorials/)
- MDN Web Docs: developer.mozilla.org/pl/
- WebAIM, skróty NVDA i VoiceOver: webaim.org/resources/shortcuts/
- „HTML Tags”, kopia W3C z 3.11.1992: [w3.org/History/19921103-hypertext/hypertext/WWW/MarkUp/Tags.html](https://www.w3.org/History/19921103-hypertext/hypertext/WWW/MarkUp/Tags.html)